A PROJECT REPORT ON

Logs Management Dashboard and Real time server logs analysis

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Varad Pundlik	Exam No: B400050556
Divyank Sagvekar	Exam No: B400050572
Parth Tagalpallewar	Exam No: B400050607
Anirudha Udgirkar	Exam No: B400050619

DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2021-2022

CERTIFICATE

This is to certify that the project report entitled

Logs Management Dashboard and Real time server logs analysis

Submitted by

Varad Pundlik	Exam No: B400050556
Divyank Sagvekar	Exam No: B400050572
Parth Tagalpallewar	Exam No: B400050607
Anirudha Udgirkar	Exam No: B400050619

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. S.P.Shintre** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. S. P. Shintre
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune
Date:

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ***'Logs Management Dashboard and Real-time Server Logs Analysis'****.*

We would like to take this opportunity to thank ***Prof. S. P. Shintre*** *for providing us with all the help and guidance we needed. We are truly grateful for his kind support and valuable suggestions.*

We are also grateful to ***Dr. G. V. Kale****, Head of the Computer Engineering Department, PICT, for his indispensable support and suggestions.*

Varad Pundlik
Divyank Sagvekar
Parth Tagalpallewar
Anirudha Udgirkar
(B.E. Computer Engg.)

ABSTRACT

When an application runs on a server, it generates logs based on various processes stored in text files. These logs are crucial for monitoring bugs, finding root causes of failures, and sharing server performance metrics with clients and partners. However, finding specific logs in Unix-based systems can be tedious.

Our proposed solution is a centralized log management platform that fetches server logs into a full-text search database in real-time, streamlining log extraction and analysis. This system improves efficiency by allowing developers to view logs separated by processes, perform root cause analysis using Large Language Models (LLMs), automate log parsing, set up alerts, enhance search capabilities, and monitor performance with real-time dashboards and machine learning analytics.

Contents

List of Figures

CHAPTER 1
INTRODUCTION

1.1 Overview

In today's cloud-native, distributed environments, logs are essential for monitoring system health, diagnosing failures, and ensuring security. However, the growing volume and variety of logs make manual analysis inefficient and outdated. Our project aims to solve this using a centralized log management system built with Elasticsearch, Filebeat, Metricbeat, and Kibana (ELK without Logstash). It collects logs and system metrics in real-time, enables fast search and visualization, and introduces LLM-based AI analysis to detect anomalies and generate alerts. The system also features self-healing scripts that automatically respond to failures, helping teams maintain system stability with less manual effort.

1.2 Motivation

Traditional log monitoring methods such as shell scripts and manual parsing struggle with scale, lack intelligence, and are time-consuming. As systems grow more complex, it's critical to have automated tools that not only collect logs but also interpret them meaningfully. This motivated us to create a platform that enhances log analysis using AI, enables proactive alerting, and provides automated fixes for common issues. Our goal was to reduce downtime, improve debugging efficiency, and enable smarter monitoring for modern infrastructure.

1.3 Problem Definition and Objectives

Problem

Monitoring large-scale applications becomes increasingly difficult when logs are scattered, unstructured, and voluminous. Traditional manual review methods and static rule-based tools often fail to detect subtle anomalies or respond quickly to emerging issues.

Objectives

- Build a centralized logging system using the ELK stack (excluding Logstash).
- Integrate AI models to classify and analyze logs semantically for meaningful insights.

- Enable real-time visualization of logs and system metrics using Kibana dashboards.
- Set up automated alerts for critical system conditions based on dynamic thresholds.
- Implement self-healing scripts that automatically fix common issues such as service crashes or high resource usage.

1.4 Project Scope & Limitations

Scope

- Collect and visualize logs and system metrics from multiple sources using Filebeat and Metricbeat.
- Use Kibana to create real-time dashboards for monitoring.
- Apply LLMs (Large Language Models) for intelligent log classification and anomaly detection.
- Trigger alert notifications when predefined thresholds or error patterns are detected.
- Execute self-healing scripts to address issues like service failure or system overload.

Limitations

- **No Logstash:** The system lacks Logstash, which limits advanced log transformation capabilities.
- **Resource Intensive:** LLM-based analysis may introduce latency for large-scale logs.
- **Static Recovery Scripts:** Self-healing actions are predefined and may not adapt to all scenarios.
- **Platform Specific:** The implementation is optimized for Linux environments.

1.5 Methodologies of Problem Solving

- **Log Collection:** Filebeat and Metricbeat are installed on target systems to gather logs and system metrics in real-time.
- **Data Indexing:** Collected data is forwarded to Elasticsearch, where it is indexed and stored for efficient querying and filtering.
- **Visualization:** Kibana is used to create dashboards that display live log streams, error trends, and resource usage patterns.
- **AI-Based Analysis:** Logs are passed through a Large Language Model (LLM) which classifies them and flags anomalies based on contextual understanding.
- **Alerting & Self-Healing:** Alerts are generated for critical errors or threshold breaches, followed by execution of pre-configured shell scripts to automatically mitigate the problem.

CHAPTER 2
LITERATURE SURVEY

The field of log management has seen rapid advancements in recent years, driven by the increasing complexity of distributed systems and the growing need for real-time observability. Traditionally, log management involved manual parsing and analysis, but modern systems now emphasize scalability, automation, and intelligence. This evolution has brought about the widespread adoption of distributed logging frameworks, cloud-native architectures, and artificial intelligence techniques that collectively enhance monitoring, fault detection, and root cause analysis.

2.1 ELK-Based Distributed Log Collection

The ELK stack—comprising Elasticsearch, Logstash, and Kibana—has become a cornerstone in open-source log analytics. Its popularity stems from its modular architecture and powerful search and visualization capabilities. A notable advancement in this domain involves integrating Apache Flume and Kafka to support distributed log collection at scale. This hybrid architecture facilitates the uniform collection and transformation of logs through a listener module, which standardizes log formats before pushing them to message brokers. Once the logs are streamed through Kafka and Redis, Logstash processes them and routes them into Elasticsearch for indexing. Kibana then provides a user-friendly interface for querying and visualizing the data in real time. This architecture enables efficient monitoring across multiple systems, making it especially valuable in microservices-based environments. Additionally, real-time alerting mechanisms can be configured to trigger automated responses based on predefined log patterns. Despite its strengths, the ELK stack can present deployment and performance challenges when managing logs in highly dynamic or high-velocity environments. Memory consumption and the need for frequent tuning of components like Logstash pipelines can become operational bottlenecks. Nevertheless, its open-source flexibility makes it a preferred choice for organizations seeking customizable and scalable log management solutions.

2.2 Centralized Log Aggregation with Splunk

Splunk offers a comprehensive, enterprise-grade solution for centralized log collection and real-time analytics. Unlike the open-source ELK stack, Splunk is a proprietary platform that integrates data ingestion, indexing, and visualization in a unified environment. It supports a wide variety of data sources including syslogs, application logs, and system metrics. With built-in intel-

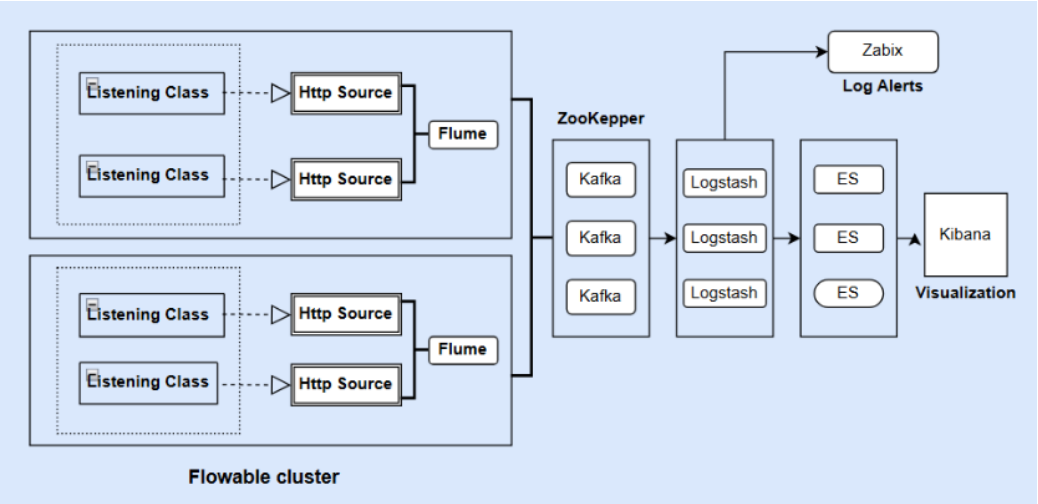

Figure 2.1: ELK-Based Distributed Log Collection

ligence and high-speed indexing, Splunk enables rapid querying and efficient correlation of logs. One key advantage of Splunk is its ability to maintain structured, searchable datasets from raw logs across large IT infrastructures. This capability simplifies tasks such as network diagnostics, user behavior analysis, and anomaly detection. However, the primary limitation of Splunk lies in its cost and lack of transparency due to its proprietary nature. The storage and licensing model can be restrictive for organizations handling massive volumes of log data over extended periods. Despite these constraints, Splunk remains a go-to choice for enterprises requiring fast deployment, minimal configuration, and integrated support for machine learning and advanced analytics.

2.3 Cloud-Native Log Collection in Kubernetes Environments

With the proliferation of containerized workloads, log management in cloud-native architectures has gained significant attention. Kubernetes, the de facto standard for container orchestration, introduces new challenges for log collection due to the ephemeral and distributed nature of containers. Modern systems address this by using sidecar containers, DaemonSets, or dedicated logging agents that capture logs directly from stdout and stderr streams. A robust Kubernetes-based log management system typically employs log rotation mechanisms like logrotate to manage disk usage and ensure con-

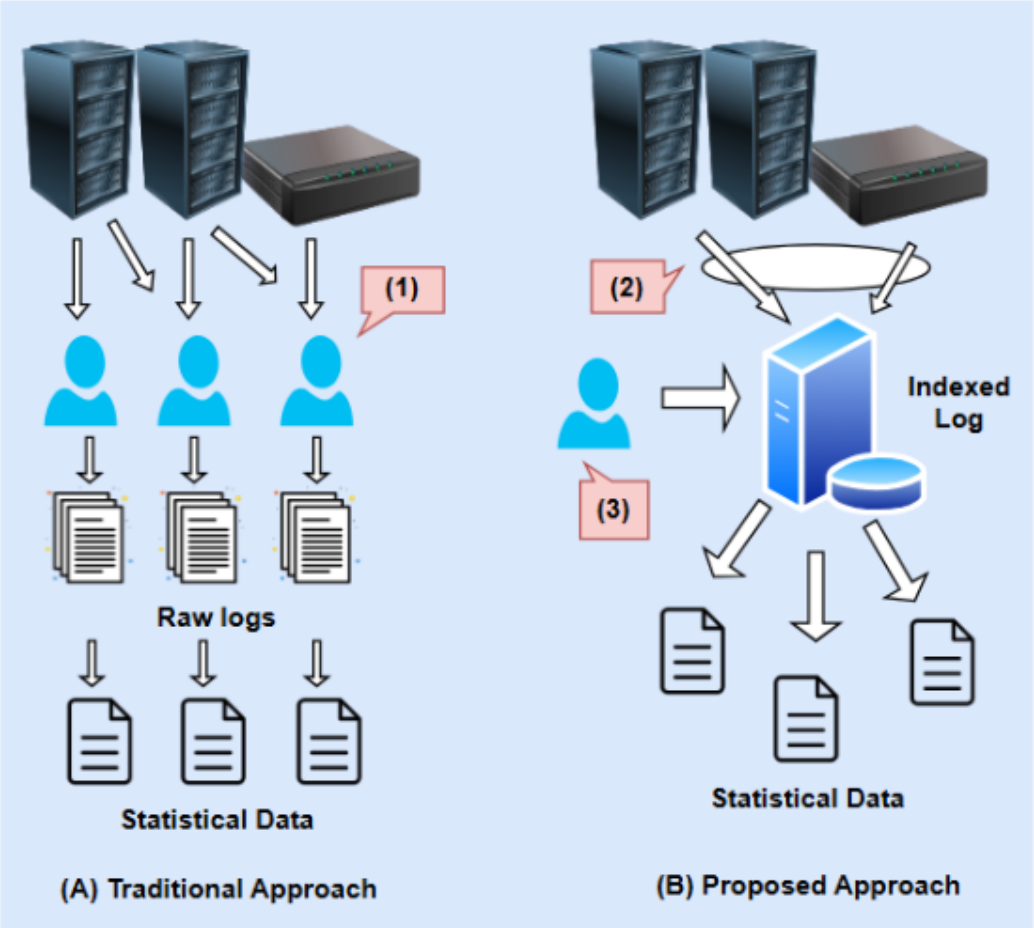

Figure 2.2: Centralized Log Aggregation with Splunk

tinuity. Logs are forwarded from individual pods to a centralized backend through lightweight agents, such as Fluentd, Filebeat, or custom-built log collectors. These systems are designed to automatically scale with the cluster and support advanced features like multi-tenant access, secure log transmission, and integration with observability platforms. This cloud-native approach enhances operational transparency and enables real-time debugging of containerized applications. The separation of logging layers also ensures that application performance remains unaffected by logging overhead. However, achieving complete observability still requires correlation with metrics, traces, and configuration events.

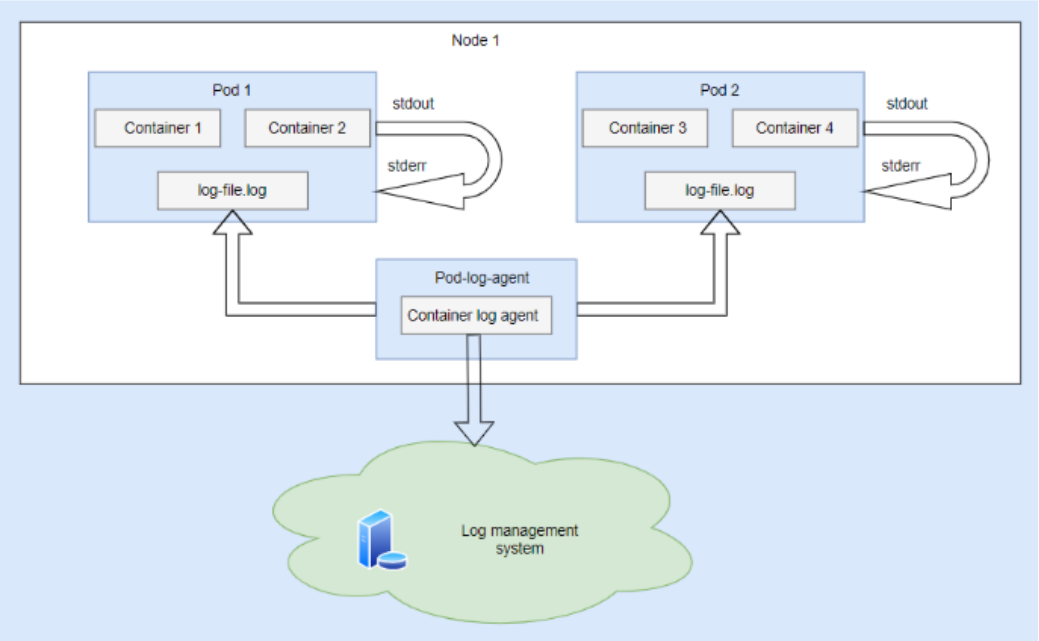

Figure 2.3: Cloud-Native Log Collection in Kubernetes Environments

2.4 Real-Time Log Monitoring and Anomaly Detection

As systems grow in complexity, real-time log monitoring becomes essential for maintaining high availability and system performance. Advanced monitoring frameworks now aggregate logs from diverse sources, such as application runtimes, operating systems, and network devices. These logs are processed using streaming analytics engines that perform continuous evaluation of data

to detect anomalies, such as latency spikes, unauthorized access attempts, or unexpected process terminations. Real-time systems are designed with low-latency pipelines that provide dashboards and alerting systems capable of reacting instantly to critical events. This reduces mean time to detect (MTTD) and mean time to recovery (MTTR), which are critical metrics in service-level agreements. However, such systems often require significant compute resources and must be carefully optimized to avoid performance degradation during peak loads. An ongoing challenge is the detection of zero-day anomalies or first-time occurrences. Traditional rule-based systems struggle with unknown patterns, which has led to the adoption of machine learning models that can learn from historical data to identify outliers and abnormal behaviors.

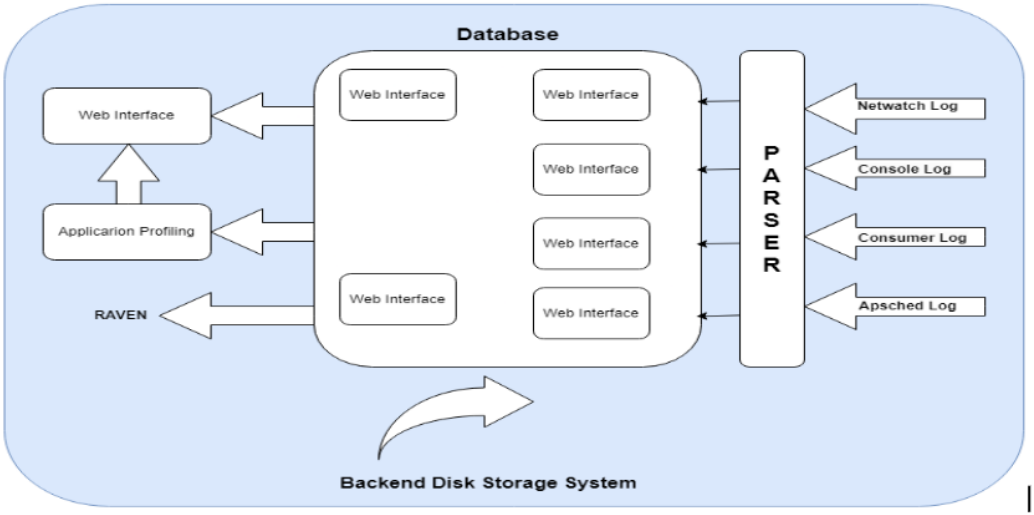

Figure 2.4: Real-Time Log Monitoring and Anomaly Detection

2.5 Deep Learning for Log Classification and Anomaly Detection

The application of deep learning to log analysis has significantly improved the efficiency and accuracy of log classification. By transforming unstructured logs into structured vector representations, deep learning models can automatically identify log types and predict potential failures. One notable approach involves the use of hybrid CNN-LSTM architectures, which combine convolutional neural networks for pattern recognition and long short-term memory networks for sequential learning. These models are capable

of processing large volumes of log data and can be trained on relatively small datasets using techniques like log tokenization, embedding (e.g., Log-Word2Vec), and data augmentation. The result is a highly accurate, automated system that significantly reduces the need for manual intervention, especially in environments where logs are generated at massive scale. Such models also allow for early detection of anomalies that might not yet have impacted system performance. The effectiveness of these systems lies in their ability to learn context over time, making them resilient to log format changes and new log event types.

2.6 Database Access Log Monitoring for Security

Specialized systems have also emerged to monitor database access logs in real time, particularly for security and compliance. These systems provide granular visibility into who accessed what data, when, and from where. By building web-based interfaces on top of MySQL or other relational databases, administrators can receive alerts for suspicious access patterns or unauthorized data manipulation. This focused monitoring approach complements broader log collection frameworks by offering deep visibility into privileged user activity and helping prevent data breaches. Real-time alerts, coupled with historical analysis, enable security teams to detect and respond to insider threats or compromised accounts more effectively.

2.7 Automation in Log Analysis for Reliability Engineering

Modern reliability engineering depends heavily on automated log analysis for predictive maintenance and incident management. Key components include log compression, parsing, and mining techniques. Compression methods such as bucket-based, dictionary-based, and statistical approaches help manage storage overhead without losing essential information. Parsing tools structure the data, making it amenable to analysis, while mining techniques reveal hidden patterns and correlations. These automation strategies are often combined with machine learning algorithms like clustering, classification, and sequence modeling to extract actionable insights. Real-time processing tools are integrated into CI/CD pipelines and DevOps workflows, enabling proactive incident resolution and enhancing system resilience. Despite these

advances, scalability and heterogeneity of log formats continue to pose challenges. Future systems are expected to incorporate adaptive parsing engines and self-learning models to dynamically adjust to new log structures.

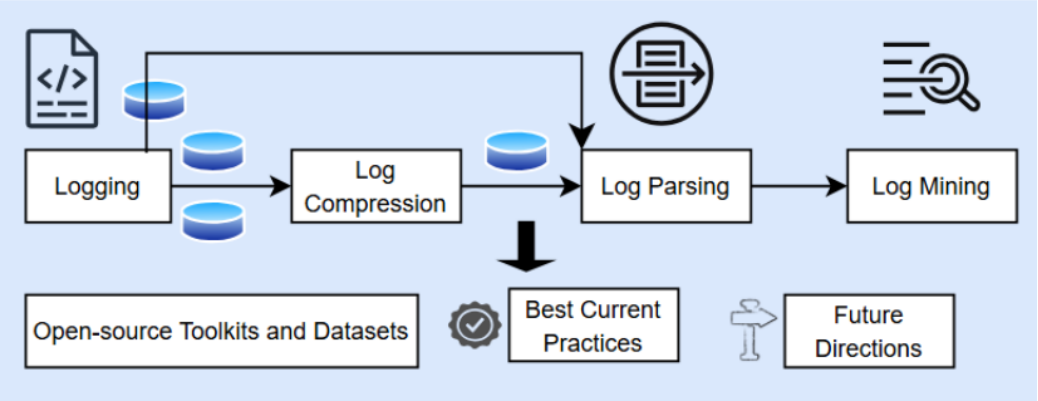

Figure 2.5: Automation in Log Analysis for Reliability Engineering

2.8 AI-Powered Root Cause Analysis (RCA) with Large Language

The latest frontier in log management involves the use of large language models (LLMs) for automated RCA. These systems synthesize logs, metrics, traces, and telemetry data to form a holistic view of system behavior. By leveraging pretrained models fine-tuned on operational data, LLMs can correlate disparate events and infer causal relationships between system components. The RCA process begins with multi-source data ingestion followed by structured analysis using AI techniques such as anomaly detection, dependency mapping, and semantic reasoning. These systems not only identify potential causes of failure but also recommend mitigation steps, effectively shortening the feedback loop in incident resolution. Unlike traditional AI systems that require extensive labeled datasets, LLMs are capable of zero-shot or few-shot learning, making them ideal for dynamic environments where new incident types emerge frequently. These models represent a major leap toward autonomous operations, where systems can self-diagnose and even self-heal based on real-time observations and historical patterns.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

This section identifies the assumptions and external dependencies relevant to the development and deployment of the LogBoard system:

- It is assumed that Filebeat and Metricbeat agents are properly installed and configured on all application servers.
- Elasticsearch cluster is available and operational for storing and querying logs and metrics.
- Admin and DevOps users have authenticated access to the dashboard for querying and configuration.
- Internet access is required to use third-party LLMs via APIs (e.g., Gemini).
- Dependencies include: Elastic Stack, FAISS for vector store, Node.js (backend), React (frontend), Jenkins (for automation), and FastAPI (LLM microservice).

3.2 Functional Requirements

3.2.1 System Feature 1: Log Collection and Indexing

Logs and system metrics will be collected in real-time using Filebeat and Metricbeat, and shipped to Elasticsearch where they will be indexed for fast retrieval and structured querying.

3.2.2 System Feature 2: LLM-Based Log Analysis

The FastAPI microservice will process logs via Retrieval-Augmented Generation (RAG) using local and API-based LLMs. It will provide root cause analysis, summarization, and operation-wise log classification.

3.2.3 System Feature 3: Alert Generation and Self-Healing

A rules-based alerting system will trigger real-time notifications (email/webhook) based on specific log events. If a failure pattern is detected, corresponding Jenkins automation jobs will be executed for self-healing.

3.2.4 System Feature 4: Dashboard and Visualization

A React-based frontend will allow authenticated users to view real-time logs, alerts, and performance metrics. It will also provide interfaces for querying RCA and automation recommendations.

3.3 External Interface Requirements

3.3.1 User Interfaces

The user interacts via a web-based dashboard built using React and hosted on Vercel. The interface displays summarized logs, metrics, root cause insights, alert status, and automation controls.

3.3.2 Hardware Interfaces

- Application server where Filebeat and Metricbeat are installed.
- Backend server (Render) for Node.js master orchestration.
- Local or cloud-based FastAPI server for LLM microservice.

3.3.3 Software Interfaces

1. Operating System: Linux-based OS (Ubuntu preferred)
2. Backend: Node.js (Express), FastAPI (Python)
3. LLM Tools: LangChain, FAISS, Ollama, Gemini API
4. Data Store: Elasticsearch 8+, MongoDB
5. Real Time Data Store: Elasticsearch 8+, MongoDB
6. Frontend: React (Vercel)
7. Automation: Jenkins API

3.3.4 Communication Interfaces

- RESTful APIs between React, Node.js backend, and FastAPI microservice.
- WebSockets or HTTP polling for live updates on alerts.

- Email and Webhook channels for alert notifications.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

The system should process log updates and RCA analysis with a response time under 3 seconds for most operations. Logs and metrics should be ingested with minimal delay (under 500ms).

3.4.2 Safety Requirements

The system must gracefully handle data loss scenarios by ensuring retries for failed log ingestion. Any automation job execution must be validated and logged.

3.4.3 Security Requirements

- Authenticated access to dashboard using role-based permissions.
- Encrypted API communications (HTTPS).
- Secure Jenkins webhook integration to avoid unauthorized automation.

3.4.4 Software Quality Attributes

- **Scalability:** System supports distributed ingestion and multi-user dashboard access.
- **Reliability:** Redundancy in Elasticsearch and error-handling in LLM pipelines ensure consistent service.
- **Usability:** Simple, clean UI for both developers and DevOps teams.
- **Maintainability:** Modular microservice-based architecture ensures ease of upgrades and bug fixes.

3.5 System Requirements

3.5.1 Database Requirements

- Elasticsearch Index for Logs

- Elasticsearch Index for Metrics
- FAISS vector store in memory (RAM-based)
- MongoDB cloud database to store metadata

3.5.2 Software Requirements (Platform Choice)

- Node.js backend deployed on Render
- FastAPI + LangChain microservice (local/cloud)
- React frontend deployed on Vercel
- Jenkins hosted CI/CD server for healing jobs

3.5.3 Hardware Requirements

- Application servers with 8 GB RAM and 4-core CPU
- Dedicated system or VM for Elasticsearch cluster with SSD storage
- LLM service recommended with GPU (for DeepSeek or Gemini fallback)

3.6 Analysis Models: SDLC Model to be applied

Agile methodology is adopted for development, with weekly sprint cycles for delivering features iteratively. Each module was built and tested independently, ensuring flexibility for improvements based on feedback. Continuous Integration and Continuous Deployment (CI/CD) is managed using Jenkins and GitHub.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The system architecture of LogBoard integrates log collection, AI-based analysis, alert generation, and automation. It utilizes ELK stack components for ingestion and storage, FastAPI-based microservice for LLM analysis, and a Node.js backend for processing and orchestration. A Vercel-hosted React dashboard presents this data to end-users in real time.

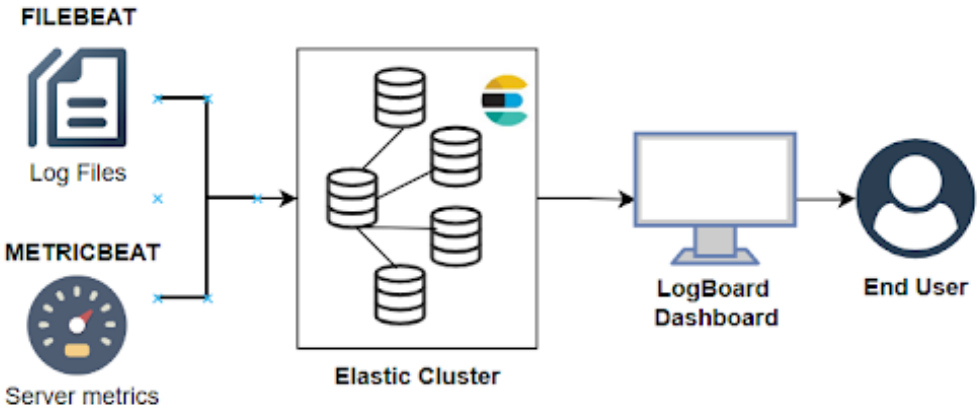

Figure 4.1: System Architecture Diagram

4.2 Mathematical Model

The system can be mathematically modeled as a tuple:

$$S = \{I, O, F, D, A, M\}$$

where:

- I is the set of inputs: $I = \{L, M_r, Q\}$ where L denotes real-time logs, M_r are metrics, and Q represents queries.
- O is the set of outputs: $O = \{S_l, R_c, A_n, H_t\}$ including log summaries (S_l), root cause outputs (R_c), alert notifications (A_n), and healing actions (H_t).
- F denotes functional modules: $F = \{f_{parse}, f_{rag}, f_{alert}, f_{heal}\}$ representing parsing, analysis, alerting, and healing.

- D is the data repository: $D = \{E_l, E_m, V_f\}$, where E_l and E_m are Elasticsearch log and metric indices, and V_f is the FAISS vectorstore.
- A is the set of actors: $A = \{U, S\}$ with U being users (Admin/DevOps) and S the system.
- M maps failures to healing jobs: $M : F_t \rightarrow J$ where F_t are failure types and J are Jenkins jobs.

The objective is to compute a function $f : I \times D \rightarrow O$ such that meaningful outputs are derived from logs and metrics with minimal latency and maximum diagnostic accuracy.

4.3 Data Flow Diagram

The Data Flow Diagram (DFD) represents the flow of data between key components of LogBoard — from log collection to alert generation and healing.

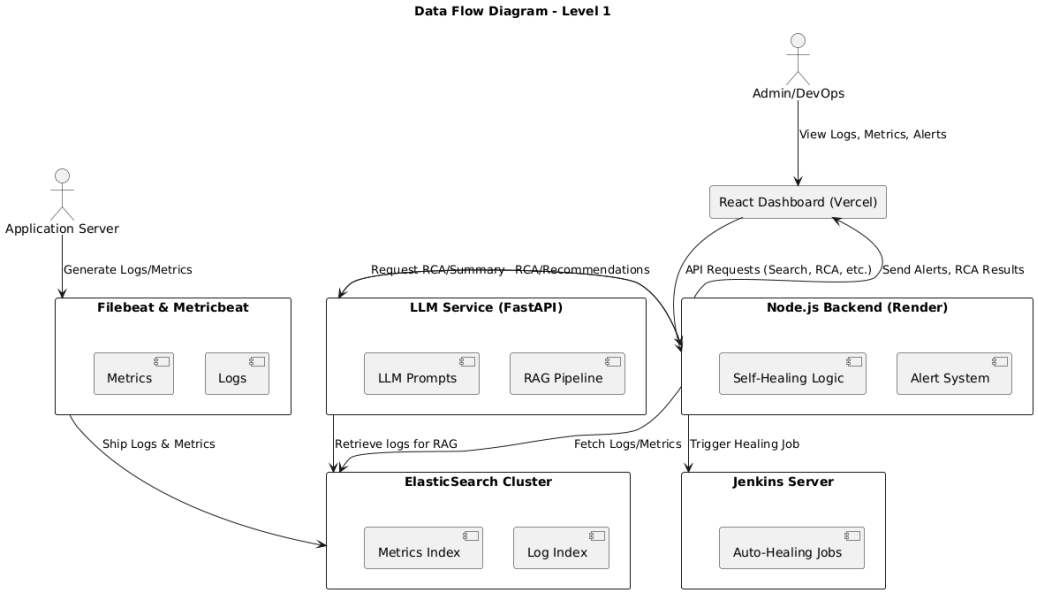

Figure 4.2: Data Flow Diagram of the LogBoard System

4.4 Use Case Diagram

This diagram illustrates how key actors (Admin/DevOps and System) interact with various subsystems of LogBoard such as LLM services, search, and

healing operations. It demonstrates system scope and user roles.

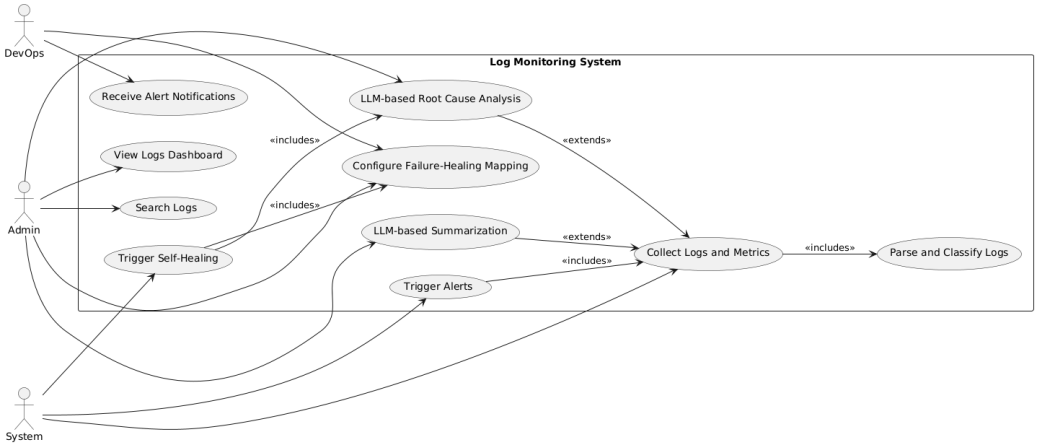

Figure 4.3: Use Case Diagram

4.5 Class Diagram

The class diagram details the system's object-oriented design. It identifies key classes such as 'LogManager', 'AlertsManager', 'Dashboard', and their relationships and responsibilities, ensuring modular and maintainable code.

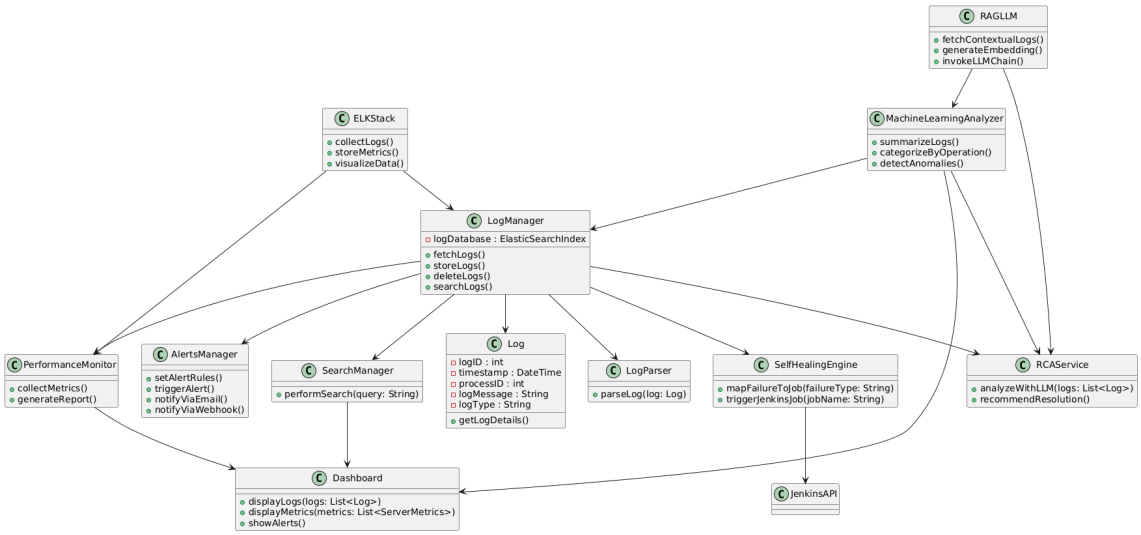

Figure 4.4: Class Diagram of the LogBoard System

4.6 Deployment Diagram

This diagram shows the physical deployment of components: ElasticSearch runs as a cluster for performance, the LLM microservice runs via FastAPI, the backend (Node.js) is deployed on Render, and the frontend React.js dashboard is hosted on Vercel. Jenkins automates healing tasks.

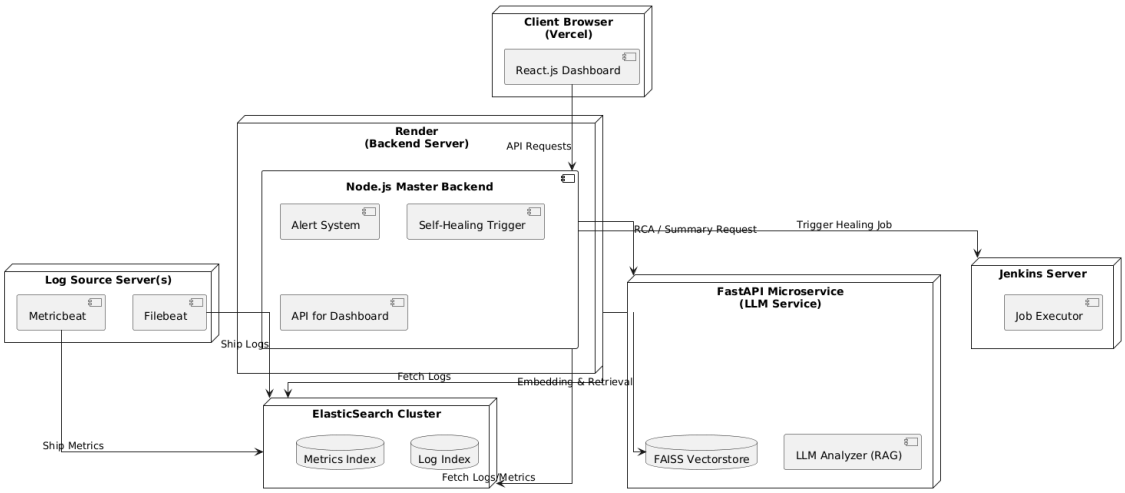

Figure 4.5: Deployment Diagram

CHAPTER 5
PROJECT PLAN

5.1 Project Estimate

This section outlines the resource estimation and effort planning for the successful implementation of the LogBoard project.

5.1.1 Reconciled Estimates

- **Development Time:** Approximately 12 weeks divided into planning, design, development, testing, and deployment phases.
- **Team Size:** 4 students, each assigned dedicated roles (frontend, backend, RCA integration, alerting).
- **Technology Stack:** ELK stack (Elasticsearch, Logstash, Kibana), React.js, Node.js, FastAPI, LangChain, FAISS, Jenkins.
- **Estimated Cost:** Mostly open-source tools used. Cost incurred mainly for cloud services (Render, Vercel), domain, and compute resource usage for LLMs.

5.1.2 Project Resources

Resources required for the development include:

- Development machines with minimum 8GB RAM and SSD storage
- Cloud hosting accounts (Render, Vercel, GitHub)
- Access to Gemini API (for LLM-based root cause analysis)
- Jenkins server or Jenkins Cloud for automation tasks

5.2 Risk Management

Effective risk management ensures timely delivery and quality assurance throughout the project lifecycle.

5.2.1 Risk Identification

- LLM response inconsistency or inaccuracies
- Integration issues between different services (FastAPI, Elasticsearch, Node.js)
- System resource limitations on local machines for running FAISS/DeepSeek models
- Potential security risks during automation triggers

5.2.2 Risk Analysis

- **High Impact:** Model inaccuracy, failed self-healing execution
- **Medium Impact:** Dashboard rendering delays, log search timeout
- **Low Impact:** Minor frontend glitches

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- Use multiple LLMs (fallback to Gemini via API if local model fails)
- Unit and integration testing for APIs between services
- Offload processing-heavy services to cloud (Render, Vercel)
- Secure Jenkins endpoints using tokens and role-based access

5.3 Project Schedule

5.3.1 Project Task Set

- Planning and Requirement Gathering
- System Design and Architecture Finalization
- Module-wise Development (Log collection, RCA, Dashboard, Alerts, Automation)
- Testing, QA, and Performance Tuning
- Final Deployment and Training

5.3.2 Task Network

- Log ingestion and RAG pipeline development must be integrated before alert generation and automation-self-healing feature
- Dashboard visualization depends on metrics and log indexing completion
- Deployment and integration testing are contingent on module completion

5.4 Team Organization

5.4.1 Team structure

- Varad Pundlik: Log Ingestion, RAG development and backend development
- Anirudha Udgirkar: Backend Development and auth feature development
- Divyank Sagvekar: Dashboard Design and Metrics ingestion
- Parth Tagalpallewar: Target Application Development and alert system.

5.4.2 Management reporting and communication

- Weekly progress meetings with project guide
- Updates shared via Google Docs and GitHub
- Communication over WhatsApp and Google Meet for day-to-day coordination

5.4.3 Timeline Chart

Phase	Tasks	Duration
Phase 1: Planning and Requirements Gathering	- Identify log sources and system requirements. - Define functional and technical requirements. - Outline architecture and technical stack.	2 weeks
Phase 2: Design and Architecture	- Design centralized log management system. - Define log ingestion, processing, and storage architecture. - Plan automation for log parsing and alerting. - Define dashboard layout and features.	3 weeks
Phase 3: Development	**3.1 Log Collection and Parsing** - Set up log collection tools (Logstash, Filebeat). - Implement automated log parsing scripts. - Configure log ingestion into Elasticsearch. **3.2 Real-Time Processing and Anomaly Detection** - Integrate Kafka for real-time log processing. - Set up machine learning models for anomaly detection. - Configure alerting mechanisms. **3.3 Root Cause Analysis (RCA) Using LLM** - Implement RCA integration with pre-trained LLM. - Set up communication between logs and RCA. **3.4 Dashboard and Visualization** - Design and develop a real-time monitoring dashboard. - Integrate performance metrics. - Implement search and filtering options.	6 weeks
Phase 4: Testing and QA	- Conduct unit and integration testing. - Perform load testing for real-time log ingestion. - Test RCA accuracy and alert mechanisms. - Fix bugs and optimize performance.	3 weeks
Phase 5: Deployment	- Deploy the system in the live environment. - Perform final system integration tests. - Train users on system usage. - Ensure smooth rollout with minimal downtime.	2 weeks
Phase 6: Monitoring and Maintenance	- Continuous monitoring of system performance. - Fine-tune machine learning models. - Update log parsing and alerting rules as needed.	Ongoing

Table 5.1: Project Timeline and Task Breakdown

CHAPTER 6
PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The **LogBoard** platform is composed of modular components that collectively enable real-time log ingestion, intelligent analysis, alerting, and automated issue resolution. The system is designed to handle the entire lifecycle of log data efficiently—from generation and processing to actionable insights and recovery.

6.1.1 Log Collection Module

This module is responsible for collecting logs and system-level metrics from various sources in real time.

- **Filebeat** is configured to monitor application log files, efficiently capturing log entries and forwarding them for processing. It supports multiline log structures, making it suitable for capturing error stack traces and detailed logs.
- **Metricbeat** is used to collect system telemetry such as CPU utilization, memory usage, disk I/O, and network throughput.
- Both tools operate as lightweight agents, forwarding logs and metrics to a centralized storage backend.

6.1.2 Data Storage and Indexing Module

Logs and metrics collected are forwarded to **Elasticsearch**, a distributed search engine optimized for log indexing and retrieval.

- All incoming logs are parsed into structured JSON documents.
- Logs are organized using **time-based indexing**, allowing for efficient searches within specific time ranges.
- This setup supports real-time querying, filtering, and correlation of logs across distributed systems.

6.1.3 Monitoring and Visualization Module

Instead of relying on Kibana, this module utilizes **custom-built dashboards or third-party visualization tools** to display logs, metrics, and alerts.

- Custom dashboards are designed to display real-time system statistics, log activity, and error trends.
- These interfaces allow developers to **filter logs by severity, time range, or source**, helping them identify root causes more quickly.
- Important metrics such as CPU usage, memory consumption, and error frequency are displayed in graphical formats for ease of understanding.

6.1.4 AI-Powered Log Analysis Module

A dedicated **microservice powered by large language models (LLMs)** performs intelligent log classification and summarization.

- The service queries logs from Elasticsearch and applies natural language processing to classify logs into categories like INFO, WARNING, ERROR, and CRITICAL.
- It also identifies **anomalous patterns** by comparing with historical logs and usage behavior.
- Long sequences of error logs are summarized into **human-readable explanations**, improving developer productivity during debugging sessions.

6.1.5 Alerting and Self-Healing Module

This module ensures the system responds to anomalies in real-time through alerts and automated corrective actions.

- **Alert conditions** are defined based on metric thresholds or specific log patterns (e.g., high CPU usage, recurring critical errors).
- When a threshold is breached or a critical pattern is detected, an **automated email alert** is triggered using **Nodemailer**.
- For predefined critical issues, the system triggers **recovery scripts** that attempt to resolve the issue automatically—such as restarting a service or clearing cache—thus improving availability and reliability.

6.2 Tools and Technologies Used

The implementation of the LogBoard platform involves a range of open-source tools and technologies that work together to ensure robust log collection, intelligent analysis, efficient storage, and automated remediation. Below is a detailed description of each tool and its role within the system.

- **Filebeat:**
  Filebeat is a lightweight log shipper developed by Elastic. It is installed on servers to monitor log files and forward them to a centralized location. In this project, Filebeat is configured to watch application log files, capture new log entries in real time, and send them directly to Elasticsearch. It supports parsing multiline logs, which is especially useful for error stack traces and detailed logs.

- **Metricbeat:**
  Metricbeat is another component from the Elastic stack used to collect system-level metrics such as CPU usage, memory utilization, disk I/O, and network activity. Metricbeat is configured to run as an agent on the host system and send telemetry data to Elasticsearch at regular intervals. This data is essential for monitoring resource health and setting up alert thresholds.

- **Elasticsearch:**
  Elasticsearch is a distributed, RESTful search and analytics engine used to store, index, and query logs and metrics. All logs collected by Filebeat and metrics from Metricbeat are sent to Elasticsearch, where they are indexed using time-series based indices. Elasticsearch supports full-text search, filtering, and aggregation operations, which are leveraged for real-time querying, correlation, and analytics.

- **Python:**
  Python is used in the development of automation scripts and AI microservices. In particular, it powers the LLM-based microservice that classifies and summarizes logs. Python's libraries such as `requests`, `transformers`, and `pandas` are used for making API requests, running NLP models, and performing data preprocessing.

- **Node.js:**
  Node.js is used to build backend services and notification logic. It is also used with `Nodemailer` for sending email alerts when critical errors or anomalies are detected in the logs or metrics. Its asynchronous

nature makes it suitable for handling I/O-bound operations like email dispatch and log processing.

- **LLM Microservice:**
  A custom microservice built using Python and large language models (LLMs) is integrated to analyze logs semantically. This service pulls logs from Elasticsearch, uses transformer-based models to classify logs by severity, detect abnormal behavior patterns, and generate summaries. It enhances the developer experience by reducing the time needed for manual inspection and root cause analysis.

- **Nodemailer:**
  Nodemailer is a module for Node.js applications to send emails. It is used to deliver real-time alerts to system administrators or developers. When a metric threshold is exceeded or a critical error is detected, an alert message is composed and sent automatically via SMTP using Nodemailer.

- **Custom Dashboard Interfaces:**
  Instead of using Kibana, custom-built dashboards or third-party visualization tools are employed to display the logs, system metrics, and alert statuses. These dashboards allow filtering by log severity, time range, and component, giving developers a focused view of the system's health and activity.

6.3 Methodology and Algorithm Details

The log management dashboard is responsible for collecting logs and metrics on real time basis, separating and summarizing logs based on distinct API operations, root cause analysis of an incident occurring, generating alerts for failures and self-healing automation to recover server and application from failure. The implementation of all these features is divided into 3 subsystems:

1. Logs and metric collection system
2. LLM based logs analysis system
3. Alert generation and self-healing automation

6.3.1 Logs and metrics collection system

The logs and metrics collection system forms the backbone of the Logs Management Dashboard. It is designed to make sure that both server logs and critical server metrics are captured, enhanced, and stored in real time, thereby providing a clear view of system performance and providing proactive error detection and resolution.

Log Collection with Filebeat:

Filebeat is a lightweight, open-source log shipper installed on every target server to continuously monitor and collect logs from specified directories. It ensures real-time log forwarding, enabling quick detection of issues. These logs are then sent to an Elasticsearch index, where they are structured for efficient searching and further processing. This seamless integration ensures that logs are not only collected but also made actionable for monitoring and troubleshooting.

Metrics Collection with Metricbeat:

Metricbeat is used to collect real-time server performance metrics, including CPU usage, memory consumption, network activity, and disk I/O. These metrics are gathered at regular intervals and sent to Elasticsearch, where they are stored in time-series indices for efficient analysis. By correlating system performance data with application logs, Metricbeat helps identify patterns and potential issues, such as performance drops leading to errors. This structured approach enables proactive monitoring, faster troubleshooting, and improved system reliability.

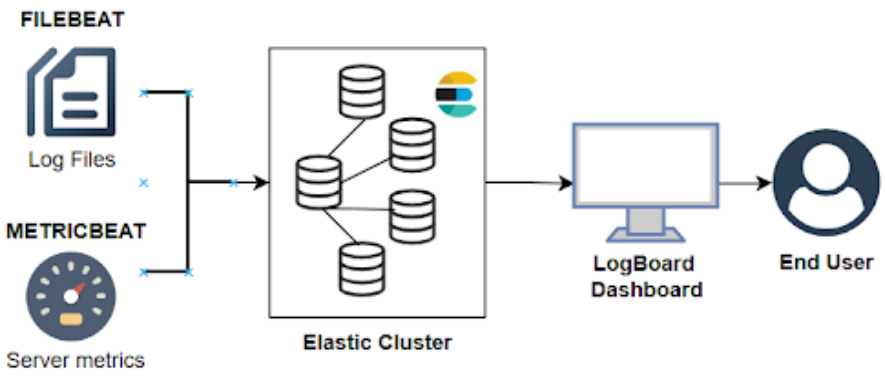

Figure 6.1: System architecture of logs and metrics collection system

6.3.2 LLM based log analysis

Log analysis features consist of Operation based log separation where log blocks are partitioned into operations, API requests and transactions where the operations and their summary is also provided based on logs.

Another LLM based logs analysis feature is Root Cause Analysis of an incident where LLM detects the incident based on real time fetched logs and collects the evidence and provides recommendations to resolve the incident.

These features are implemented on a dedicated FastAPI based microservice using RAG (Retrieval Augmented Generation) and LangChain with the help of various LLMs like tiny-llama, DeepSeek-R1(1.5b) and gemini-1.5-pro. In which tiny-llama and deepseek were loaded locally using Ollama and gemini was used with the help of API.

Retrieval Augmented Generation (RAG) is a method to leverage applications of LLM based on a custom data to generate response relevant to specific needs. LLM is trained on last public data and its capabilities can be extended using RAG by training it on internal data or knowledge base (logs and metrics in this case).

LangChain is a python framework used to build AI agents. It provides modular components for retrievers, LLMs, and vector stores, making it easy to build RAG-based pipelines.

During the implementation the logs from Elasticsearch endpoint and index were fetched using Elasticsearch client in Python language and the logs were embedded in FAISS in memory vectorstore using "all-MiniLM-L6-v2" embedding model available on Huggingface. Vectorstores are representations of data in a vector space model which makes data retrieval easier for RAG based pipelines.

Then the deepseek-r1:1.5b model was trained on the logs in the vectorstore which was loaded locally using Ollama and separate prompt templates were created for each feature with the context of the logs retriever.

Finally API routes were added for both features and separate chains were invoked for each route with trained model and prompt template.

Algorithm:

1. Initialize Components: Connect to Elasticsearch, load API key, initialize embedding model, and set up LLM.

2. Fetch Logs: Retrieve logs from Elasticsearch, convert them into Document objects.

3. Generate Embeddings: Use HuggingFaceEmbeddings to vectorize logs and store them in FAISS.

4. Retrieve Relevant Logs: Query FAISS to find similar logs (k = logs.size()).
5. Apply LLM Prompts: Use predefined templates for summarization, root cause analysis, and automation mapping.
6. Execute LLM Queries: Run RetrievalQA, parse structured JSON output.
7. Store and Update Results: Maintain latest_results, periodically fetch and process logs.
8. Expose API Endpoints: Provide FastAPI routes for summary, root cause, and automation retrieval.
9. Background Processing: Continuously update logs in separate threads every 3 minutes.

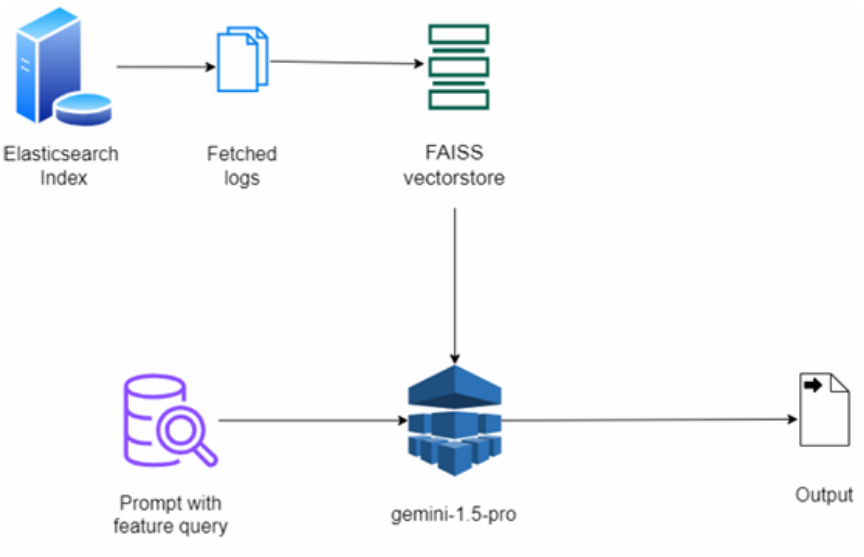

Figure 6.2: System architecture of training LLM using RAG

6.3.3 Alert generation and self-healing automation

The alerting mechanism in our Logs management Dashboard performs an important role in making sure timely notifications for critical events. Through

leveraging NodeMailer with Node.js, we have carried out an automated alerting system that detects anomalies and predefined situations in log data, permitting proactive troubleshooting.

NodeMailer is an open-source email-sending utility used to provide automated email notifications for significant events. It supports real-time notification by integrating with SMTP or third-party email providers to provide timely alert delivery for system crashes, security incidents, or anomaly detection within log data.

It makes use of a rules-based technique for flexible alert configurations, along with keyword matches, frequency-based signals, and anomaly detection. Node.js fetches logs at regular intervals and identifies problems like errors, spikes, authentication failures, or system downtime.

When an alert condition is met, it notifies through various channels inclusive of email, Slack, webhooks, or custom integrations, ensuring a timely incident response.

Self-healing are the automation tasks which are triggered when an incident occurs where a client can dynamically add mappings of failure and automation tasks (preferably Jenkins job) to correct the failure.

Then based on the logs and metrics collected in Elasticsearch, the mapping of error and healing job can be passed to the RAG system along with the logs and metrics and then the exact failure can be detected. Then the corresponding automation job is triggered from LogBoard Backend server using Jenkins API.

During this implementation 6 default cases were considered:

Sr.No	**Failure Type**	**Self-Healing Job**
1	Application crash	ApplicationRestart
2	ECONNREFUSED	DBRestart
3	Could not connect to database	DBRestart
4	DiskFull	ScaleUp
5	Unauthorized access	BlockSuspiciousIP

Table 6.1: Mapping of failure type and Auto-Healing Job

CHAPTER 7
SOFTWARE TESTING

7.1 Type of Testing

To ensure the reliability and robustness of the LogBoard system, various types of software testing methodologies were employed throughout development:

- **Unit Testing:** Each module such as the Log Parser, Alert System, and LLM Analyzer was independently tested using unit tests to verify the correctness of individual components.
- **Integration Testing:** Integration tests ensured proper communication between Filebeat/Metricbeat, ElasticSearch, FastAPI-based LLM services, and the Node.js backend.
- **System Testing:** The entire system was tested end-to-end to validate all functional requirements including log collection, analysis, alerting, and automation.
- **Load Testing:** Historical OpenSSH logs were used to simulate high log volume and test the performance and stability of the RAG pipeline and ElasticSearch cluster.
- **Regression Testing:** After major updates (e.g., integration of new LLM models or changes in API endpoints), regression testing was performed to ensure existing functionality remained unaffected.
- **User Acceptance Testing (UAT):** Final testing was conducted in collaboration with stakeholders (e.g., DevOps users) to validate whether the dashboard met usability and operational expectations.

7.2 Test Cases Test Results

The following summarizes key test cases and their observed results during the implementation and validation phases.

- **TC1: Log Parsing Accuracy Input:** Sample structured and unstructured logs **Expected Output:** Logs are correctly parsed and classified by type **Result:** *Pass*
- **TC2: Root Cause Analysis via LLM**
  Input: Logs representing known incidents

Expected Output: Correct root cause and recommendation generated
Result: *Partial Pass* (Gemini model passed, DeepSeek was generic)

- **TC3: Alert Triggering on Anomalies**
  Input: Simulated failed authentication logs
  Expected Output: Email notification triggered to DevOps
  Result: *Pass*

- **TC4: Automation Job Mapping and Execution**
  Input: ECONNREFUSED logs with mapped Jenkins job
  Expected Output: Appropriate self-healing job executed
  Result: *Pass*

- **TC5: Dashboard Rendering of Logs and Metrics**
  Input: Real-time logs and metrics from ElasticSearch
  Expected Output: Logs and metrics rendered correctly in UI
  Result: *Pass*

All results demonstrate the functionality and resilience of the LogBoard platform, ensuring that it is production-ready and robust against real-world logging and monitoring challenges.

7.3 Load Testing Results

Apart from application development, operational features like incident detection are crucial for a system in this use case. Hence 2000 historical logs of OpenSSH systems were provided to LogBoard for Root Cause Analysis and detecting incidents from the logs it is also crucial from stress testing perspective to analyze such a large number of logs. The logs consisted of operations like valid or invalid login and logout and logging of authorized and unauthorized access of system.LogBoard detected various root causes and provided their resolutions as below:

Sr. No	Root Cause	Recommendation
1	Brute-force SSH attacks from various IPs targeting root and other common usernames (admin, user, support, etc.) including invalid user attempts.	Implement stricter SSH security measures. Block or rate-limit suspicious IP addresses. Disable password authentication for root and use key-based authentication. Configure fail2ban to automatically ban IPs after repeated failed login attempts. Consider using a port other than the default 22.
2	Failed reverse DNS lookups for some connecting IPs, indicating possible break-in attempts.	Investigate the IPs with failed reverse DNS lookups. While not conclusive proof of malicious intent, it warrants further scrutiny. Consider blocking these IPs if they exhibit other suspicious behavior.
3	Too many authentication failures leading to disconnections.	This is a consequence of brute-force attempts. Address the root cause of the brute-force attacks as recommended above.
4	Connection reset by peer during SSH authentication from 183.62.140.253.	Check network connectivity between the client and server. This could be a transient network issue or a sign of the client abruptly terminating the connection.

Table 7.1: Root Cause Analysis with Corresponding Recommendations

CHAPTER 8
RESULTS

8.1 Results

To evaluate the effectiveness of log and metric management, sample logs were collected from a Simple Note Application, which is a MERN (MongoDB, Express.js, React.js, Node.js) stack-based web application designed for note-taking. This application includes essential functionalities such as Create, Read, Update, and Delete (CRUD) operations along with user authentication features. The application was deployed in a controlled environment to simulate real-world user interactions and generate relevant operational logs.

For real-time log collection, Filebeat, a lightweight shipper for forwarding and centralizing log data, was configured on the server where the application was running. Filebeat continuously monitored application log files and forwarded them to Elasticsearch, a powerful search and analytics engine. These logs were indexed and stored in a dedicated Elasticsearch index, allowing for efficient search, filtering, and analysis.

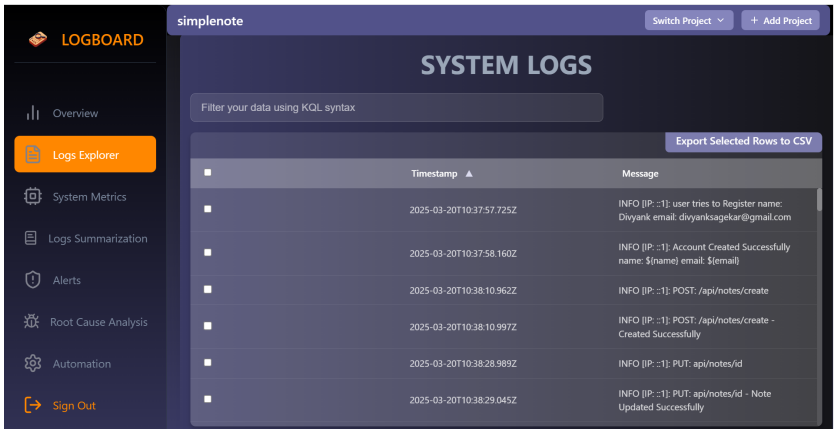

Figure 8.1: Logs saved on Elasticsearch (Simple Note App)

In addition to logs, performance metrics were also collected to monitor the system's resource utilization. Metrics such as CPU usage, memory usage, and network usage were gathered from the application's host system. These metrics were then visualized using the LogBoard web interface, a custom dashboard built to provide real-time insights into system performance.

The combination of logs and metrics provided a holistic view of the application's health and operational status. This integrated observability setup not only facilitated real-time monitoring but also enabled root cause analysis in case of performance degradation or unexpected behavior.

This figure displays CPU usage metrics captured from the host system in real time. Monitoring this metric allows tracking application processing

Figure 8.2: CPU Usage Metrics on Elasticsearch

load and detecting bottlenecks.

Figure 8.3: Memory Usage Metrics on Elasticsearch

This figure illustrates memory usage trends to detect potential memory leaks and observe how system resources are managed over time.

This figure presents real-time network throughput (inbound/outbound) of the server, helping identify traffic patterns and network-related anomalies.

To enhance the observability and intelligence of log analysis, the previ-

Figure 8.4: Network Throughput Metrics on Elasticsearch

ously collected logs from the Simple Note Application were processed using the DeepSeek-R1, a powerful open-source Large Language Model (LLM). The model was utilized to demonstrate two advanced log management features:

1. **Log Summarization** The LLM was able to parse large volumes of logs and generate concise summaries that capture key events, trends, and anomalies. This drastically reduces the time needed for manual log inspection and allows developers and system administrators to quickly grasp the system's behavior over a given period

2. **Operation-wise Categorization** In addition to summarization, the LLM categorized logs based on the type of operation, such as user login events, note creation or updates, system warnings, and errors. This structured categorization enables efficient filtering, faster troubleshooting, and better understanding of the application's usage patterns.

The outputs from DeepSeek-R1 demonstrate how LLMs can significantly improve log interpretation by providing human-readable insights from raw log data.

Using DeepSeek-R1, logs were summarized and categorized into operations such as login, note creation, errors, and warnings. This greatly enhanced readability and analysis speed.

Log Summarization: Captures major trends and anomalies in large volumes of logs quickly.

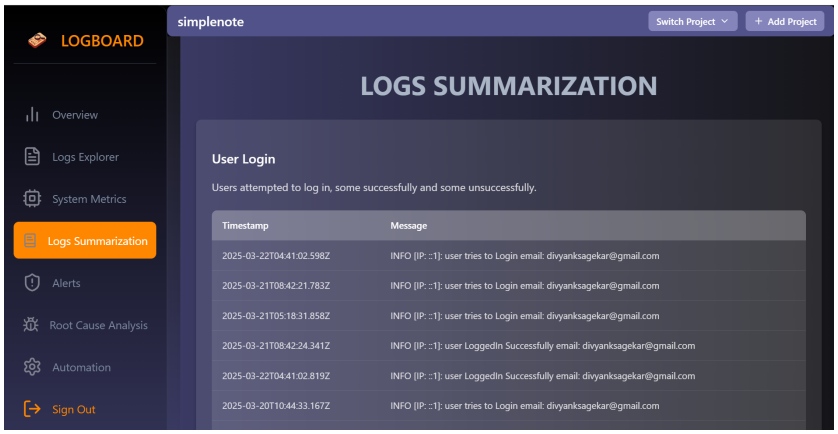

Figure 8.5: Operation-Wise Log Categorization and Summary

Categorization: Groups logs by activity type, improving search and root cause tracing.

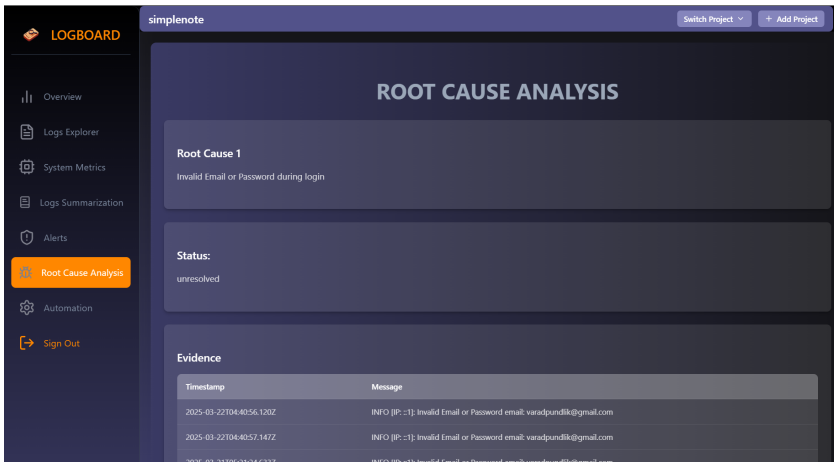

Figure 8.6: Root Cause Analysis Output

As part of the intelligent log analysis pipeline, the Root Cause Analysis (RCA) feature was implemented using the DeepSeek-R1 LLM. This feature aims to automatically identify and diagnose system failures or issues by analyzing log patterns and contextual information. The output of the RCA process is structured into four key components:

- **Failure**:
  This section identifies and describes the specific error, anomaly, or un-

expected behavior detected in the logs. It highlights what went wrong in the application or system.

- **Status**:
  It provides an overview of the current state of the system or component associated with the failure, indicating whether the issue is ongoing, resolved, or intermittent.
- **Evidence**:
  The evidence includes relevant log snippets or metrics that support the diagnosis. These serve as proof of the failure and help in understanding the sequence of events leading up to the issue.
- **Recommendation**:
  Based on the analysis, the model suggests actionable steps or best practices to resolve the problem and prevent future occurrences. These recommendations can range from configuration changes to code-level fixes.

This structured RCA output enhances the system's self-diagnosis capabilities and empowers developers and system administrators to perform faster, data-driven troubleshooting.

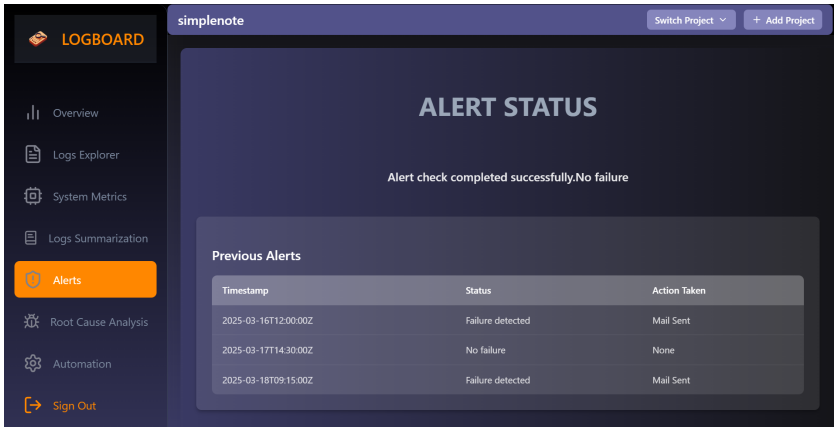

Figure 8.7: Alert Trigger Output

To ensure timely responses to critical issues, an **alerting system** was integrated into the log management pipeline. This system monitors logs in real time and triggers alerts based on predefined conditions or thresholds. The alert mechanism is designed to notify system administrators or relevant stakeholders about potential issues, such as application errors, performance degradation, or unusual activity.

A **web interface** was developed to configure and manage these alerts, providing users with an intuitive dashboard for setting alert criteria, such as:

- **Error levels** (e.g., critical, warning, informational)
- **Threshold limits** for resource utilization (e.g., CPU or memory usage spikes)
- **Patterns or anomalies** based on log content (e.g., frequent authentication failures or failed operations)

Once an alert is triggered, an **email notification** is automatically sent to the specified recipients, ensuring that the right people are informed in real-time. This proactive alerting system helps prevent issues from escalating and enables faster response times, reducing downtime and improving system reliability.

The integration of alerts with email notifications ensures that critical events are quickly addressed, enhancing the operational efficiency of the application.

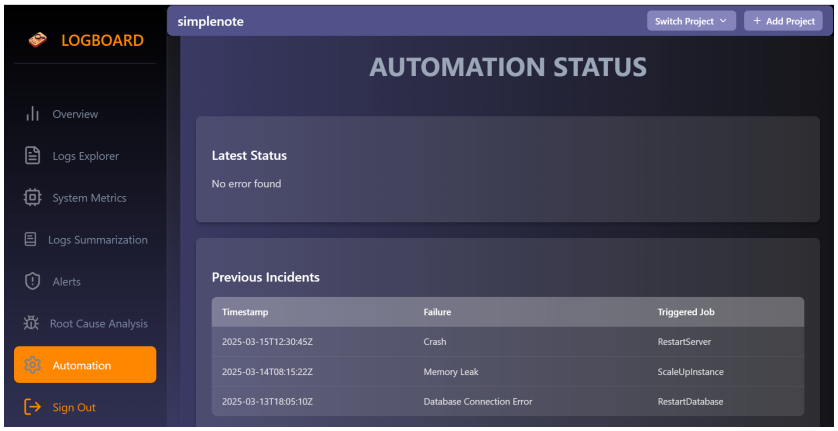

Figure 8.8: Automation and Self-Healing Output via Jenkins

To enhance the system's self-healing capabilities, an automated response system was implemented based on the recommendations provided by the DeepSeek-R1 LLM during Root Cause Analysis (RCA). When the LLM identified a failure condition in the logs, it automatically suggested an appropriate remediation action. Based on these recommendations, an automation job was triggered using the Jenkins API. Jenkins, a widely used automation server, was configured to run specific jobs that could address the identified issue. These jobs could include tasks like restarting services,

deploying patches, or reconfiguring system settings, depending on the nature of the failure. The automation job was designed to initiate without human intervention, minimizing response time and reducing manual troubleshooting efforts.

8.1.1 LLM Performance Comparison in RAG Pipeline

As part of the evaluation of the log analysis and automation system, a performance comparison was conducted across multiple Large Language Models (LLMs) used within the RAG (Retrieve and Generate) pipeline. The LLMs were assessed based on factors such as:

- Log Summarization Accuracy: How effectively each model summarized the log data.
- Root Cause Analysis Precision: The ability of each LLM to accurately identify and categorize failure conditions.
- Recommendation Quality: The relevance and usefulness of the recommended remediation actions.
- Automation Trigger Success: How reliably the automation jobs were executed based on the LLM's output.

This comparison helped in determining which LLM performed best in terms of accuracy, efficiency, and overall system effectiveness.

Sr. No.	LLM	Interface	Performance Remark
1	tiny-llama	Ollama	Unable to produce output in the provided schema.
2	DeepSeek-r1:1.5b	Ollama	Generated output in schema but the output was incorrect and generic.
3	gemini-1.5-pro	API	Generated output in the provided schema and output was correct.

CHAPTER 9
CONCLUSIONS

9.1 Conclusions

1. The proposed Logs Management Dashboard successfully integrates real-time log collection using Filebeat and Metricbeat with Elasticsearch, providing scalable and efficient data ingestion and storage.

2. The architecture eliminates the need for Logstash while maintaining structured log processing, enhancing system performance and simplifying deployment.

3. A dedicated FastAPI-based microservice enabled LLM-based log analysis for root cause detection and summarization using Retrieval-Augmented Generation (RAG).

4. Among tested models, Gemini-1.5-Pro delivered the most accurate structured output for incident diagnosis, validating the importance of model scale and capability in production systems.

5. The system demonstrated robust automation by mapping failure patterns to Jenkins-triggered self-healing tasks.

6. Alerts were reliably generated via Nodemailer and integrated with the web interface, contributing to proactive issue resolution and minimized downtime.

9.2 Future Work

1. Fine-tuning the LLMs on domain-specific logs can improve detection accuracy.

2. Integration of adaptive self-healing strategies through reinforcement learning can further enhance automation and fault recovery.

3. Expansion to cross-platform support and container-native environments like Kubernetes would broaden system applicability.

9.3 Applications

1. The system can be deployed in enterprise DevOps pipelines to reduce MTTR through intelligent log analysis and automation.

2. It is suitable for infrastructure monitoring, security auditing, and real-time root cause diagnostics across distributed systems.

3. Educational institutions and research environments can leverage it for log analysis in lab-scale experiments and cybersecurity simulations.

CHAPTER 10
APPENDIX

10.1 Appendix A

Problem Statement Feasibility Analysis

In assessing the feasibility of the problem statement, it is crucial to determine the computational complexity of the log parsing and root cause analysis processes. Using **satisfiability analysis**, the system can be evaluated to see if certain constraints (e.g., server load, memory usage, or real-time processing speeds) are met for a given dataset.

The problem of log analysis, especially anomaly detection and root cause analysis, can often be framed as NP-Hard or NP-Complete depending on the complexity of the dataset and algorithms used. For instance:

- **NP-Hard Problems**: Tasks like real-time anomaly detection in large datasets may belong to the NP-Hard class due to their dependence on pattern recognition and the need for non-deterministic approaches.
- **NP-Complete Problems**: Certain log parsing and classification problems may be solvable in polynomial time using specific algorithms but verifying the solution might require more computational power, classifying them as NP-Complete.
- **P-Type Problems**: Basic log fetching and parsing tasks that can be automated via scripts (e.g., using predefined regular expressions) typically fall into the P class as they can be solved efficiently in polynomial time.

By employing **modern algebraic models** such as graph theory, set theory, and matrix algebra, complex log relationships and dependencies can be represented and analyzed more efficiently. Furthermore, using algorithms based on these models, developers can explore feasible solutions that optimize system performance while staying within acceptable resource limits.

10.2 Appendix B

10.2.1 Survey Paper Details:

- Title: Literature Review on Logs Management Dashboard and Real-Time Server Logs Processing
- Journal: PICT's International Journal of Engineering and Technology (PIJET)
- Status: Published
- URL: `https://pijet.org/papers/volume-2%20issue-1/Final%20Revised%20Paper_Pijet-10_Done.pdf`

10.2.2 Implementation Paper Details:

- Title: Enhancing Log Monitoring with ELK, LLM-Based Analysis, Automated Alerts, and Self-Healing Mechanisms
- Conference: 10th International Conference Information,Communication Computing Technology (ICICCT-2025)
- Status: Submitted

10.3 Appendix C

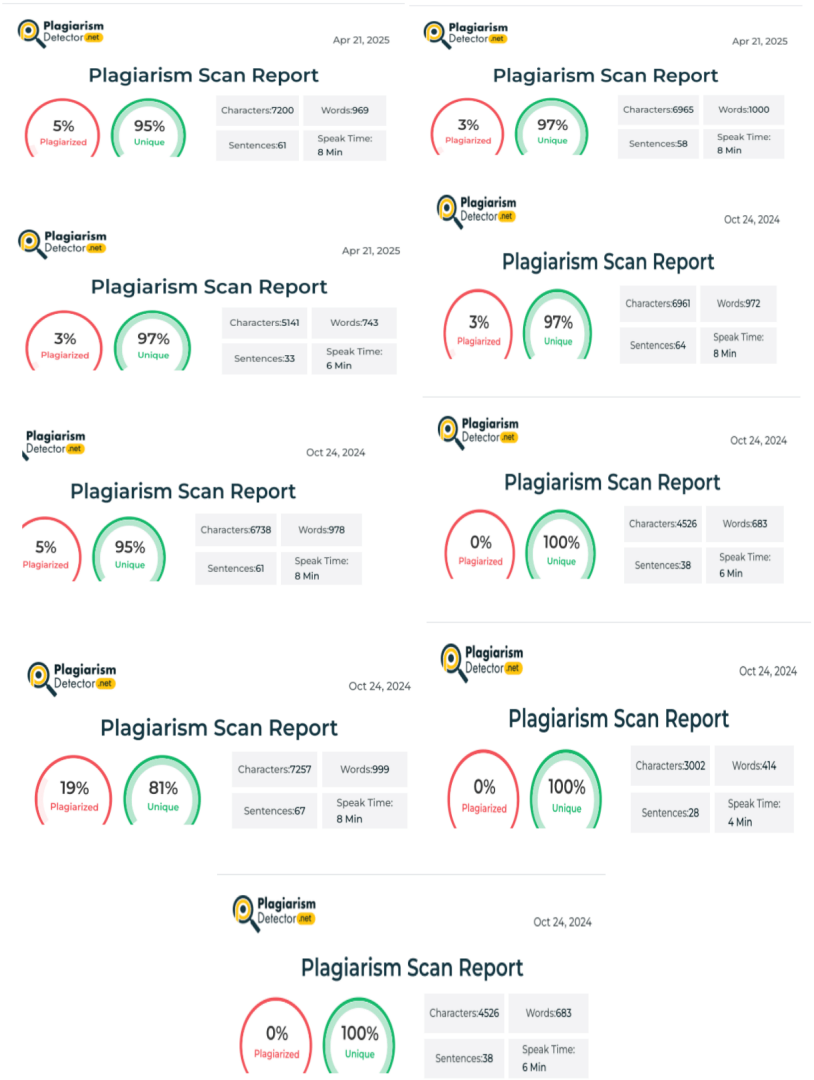

Figure 10.1: Plagiarism Scan Reports

Net Plagiarism: $\tilde{4}.95\%$

CHAPTER 11
REFERENCES

[1] Y. Zong, *Distributed log collection for business processes based on ELK architecture*, 2023 IEEE 2nd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA), Changchun, China, 2023, pp. 1523–1529.

[2] Masaru Okumura and Sho Fujimura, *Constructing a Log Collecting System using Splunk and its Application for Service Support*, In Proceedings of the 2016 ACM SIGUCCS Annual Conference (SIGUCCS'16), ACM, New York, NY, USA, 2016, pp. 103–106.

[3] T. Chen, H. Suo and W. Xu, *Design of Log Collection Architecture Based on Cloud Native Technology*, 2023 4th Information Communication Technologies Conference (ICTC), Nanjing, China, 2023, pp. 311–315.

[4] Raghul Gunasekaran, Sarp Oral, David Dillow, Byung Park, Galen Shipman, Al Geist, *Real-Time System Log Monitoring/Analytics Framework*, Oak Ridge National Laboratory.

[5] A. Setiyadi and E. B. Setiawan, *Information System Monitoring Access Log Database on Database Server*, Universitas Komputer Indonesia, Bandung, Indonesia.

[6] S. Ramachandran, R. Agrahari, P. Mudgal, H. Bhilwaria, G. Long, A. Kumar, *Automated Log Classification Using Deep Learning*, Procedia Computer Science, Volume 218, 2023, Pages 1722–1732.

[7] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, Michael R. Lyu, *A Survey on Automated Log Analysis for Reliability Engineering*, ACM Comput. Surv., June 2021.

[8] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, et al., *Automatic Root Cause Analysis via Large Language Models for Cloud Incidents*, EuroSys '24, ACM, 2024, pp. 674–688.

[9] Naeem Akhter, Muhammad Idrees, Furqan-ur-Rehman, *Sorting Algorithms – A Comparative Study*, International Journal of Computer Science and Information Security, 14(12):930–936.

[10] Jeanderson Candido, Mauricio Aniche, Arie van Deursen, *Log-based Software Monitoring: A Systematic Mapping Study*, Journal of Systems and Software, 2022.

[11] G. Gayatri Likhita, P. K. Sahoo, *Log Management In Cloud Through Big Data.*

[12] Nicolas Aussel, Yohan Petetin, Sophie Chabridon, *Improving Performances of Log Mining for Anomaly Prediction Through NLP-based Log Parsing*, IEEE MASCOTS.

[13] D. Cappelli, A. P. Moore, R. F. Trzeciak, *Logging and Log Management: The Authoritative Guide*, 2016.

[14] Van-Hoang Le, Hongyu Zhang, *Log Parsing with Prompt-based Few-shot Learning*, University of Newcastle and Chongqing University.

[15] Arie van Deursen, *Tools and Benchmarks for Automated Log Parsing.*

[16] M. A. Casanova, Y. Tan, *Data Storage*, Proceedings of the 2020 International Conference on Data Science and Advanced Analytics (DSAA), pp. 11–22.

[17] Ying Fu, Meng Yan, Zhou Xu, Xin Xia, Xiaohong Zhang, Dan Yang, *An Empirical Study of the Impact of Log Parsers on the Performance of Log-based Anomaly Detection.*

[18] M. A. Casanova, Y. Tan, *Data Storage (Duplicate Entry)*, Proceedings of the 2020 International Conference on Data Science and Advanced Analytics (DSAA), pp. 11–22.

[19] J. P. A. Almeida, E. Silva, *System Log Parsing: A Survey*, Journal of Systems and Software, 211, 111256, 2023.

[20] Pinjia Heu, Jieming Zhu, *An Evaluation Study on Log Parsing and Its Use in Log Mining.*

[21] Shijie Zhang, Gang Wu, *Efficient Online Log Parsing with Log Punctuations Signature.*

[22] G. A. Pereira, C. M. A., K. M., *Log-based Software Monitoring: A Systematic Mapping Study*, Journal of Systems and Software, 190, 111215, 2022.

LIST OF ABBREVIATIONS

Abbreviation	Illustration
ELK	Elasticsearch, Logstash, Kibana
LLM	Large Language Model
RCA	Root Cause Analysis
RAG	Retrieval-Augmented Generation
UI	User Interface
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
FAISS	Facebook AI Similarity Search

LIST OF FIGURES

LIST OF TABLES